



Support Vector Machines (SVM)

Aprendizaje Supervisado · Clasificación con Margen Máximo · Detección de Fraude con Tarjetas

Objetivos de Aprendizaje

Al finalizar esta clase serás capaz de:

1 Explicar qué es el hiperplano, el margen y los vectores de soporte, y por qué SVM maximiza el margen.

2 Describir el efecto de los hiperparámetros C y γ , y elegir entre los kernels lineal, polinómico y RBF.

3 Entrenar y evaluar modelos SVC con Pipeline, escalado y `class_weight='balanced'` en un dataset desbalanceado.

4 Reentrenar el modelo final con el 100% de los datos, exportarlo con joblib y usarlo con datos nuevos.

¿Qué es una Support Vector Machine?

Un clasificador que busca la frontera más segura entre dos clases

Definición: una SVM (Support Vector Machine, Máquina de Vectores de Soporte) es un algoritmo de aprendizaje supervisado que encuentra el hiperplano que separa dos clases con el **mayor margen posible**.

Características clave

- Se usa sobre todo para **clasificación**.
- Maneja fronteras lineales y no lineales mediante **kernels**.
- Funciona bien con datasets pequeños y medianos.
- Robusta cuando hay muchas variables.

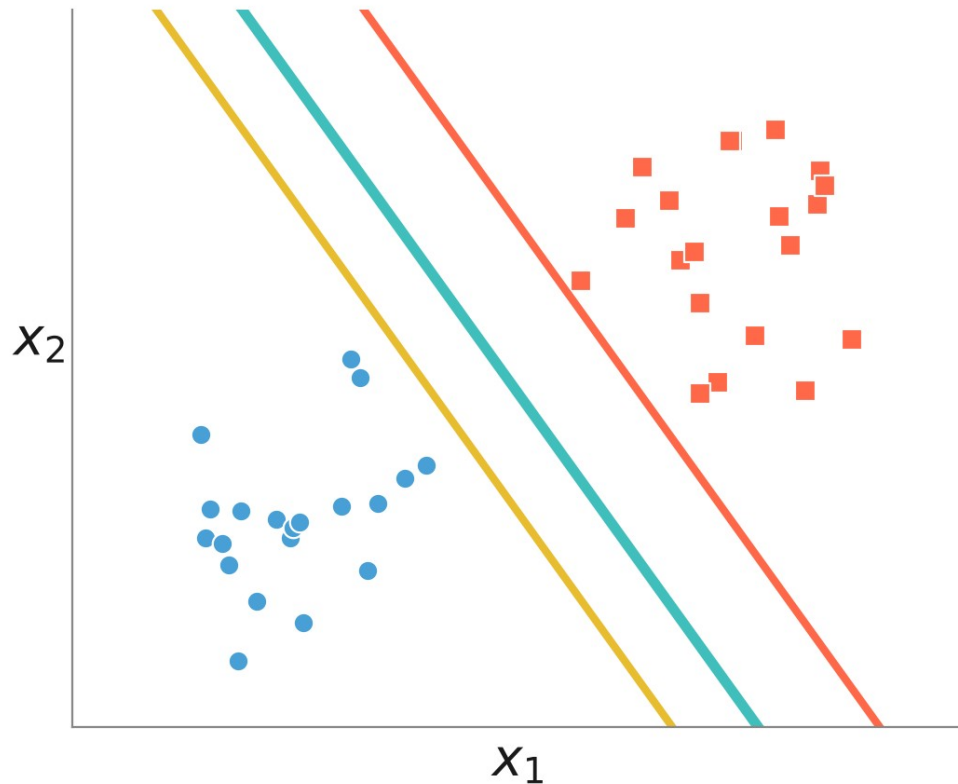
Origen

- Propuesta por **Corinna Cortes y Vladimir Vapnik** en 1995.
- Artículo: Support-Vector Networks, revista Machine Learning, vol. 20.
- Fue el clasificador dominante antes del auge del deep learning.

Fuente: Cortes, C. & Vapnik, V. (1995). Support-Vector Networks. Machine Learning, 20(3), 273–297.

El Problema: Muchas Fronteras Posibles

Si los datos son separables, existen infinitas líneas que los separan sin errores · azul = clase A, coral = clase B

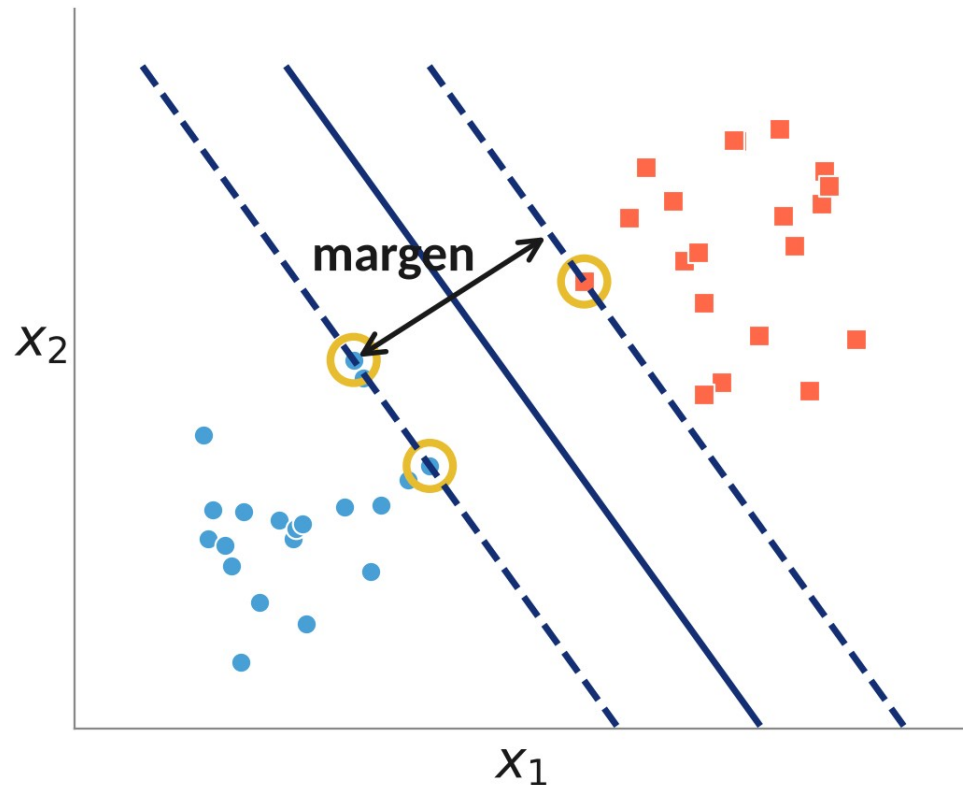


- Mala: casi toca la clase B
- Aceptable: cerca de la clase A
- Óptima (SVM): margen máximo

- Las tres líneas clasifican **perfecto** los datos de entrenamiento.
- Una línea pegada a una clase falla con datos nuevos que caigan un poco más allá.
- La Regresión Logística puede terminar en cualquiera de ellas.
- **SVM elige siempre la línea más alejada de ambas clases:** la de margen máximo.
- Más distancia a los datos = más tolerancia a datos nuevos.

El Hiperplano y el Margen

Los dos conceptos centrales de SVM



-  Vectores de soporte
-  Hiperplano $w \cdot x + b = 0$
-  Bordes del margen

Hiperplano

- Frontera que separa las clases.
- En 2D es una línea; en 3D, un plano.
- w : vector perpendicular; b : desplazamiento.

$$w \cdot x + b = 0$$

Margen

- Distancia entre el hiperplano y los puntos más cercanos de cada clase.

Vectores de Soporte

Los puntos que dan nombre al algoritmo

Definición: los vectores de soporte son los puntos de entrenamiento que quedan **sobre el borde del margen o dentro de él**. Son los únicos que determinan dónde queda el hiperplano.

Solo ellos cuentan

- Si se borran los demás puntos y se reentrena, el hiperplano no cambia.
- El modelo guarda solo estos puntos.

Sensibles a outliers

- Un punto atípico cerca de la otra clase puede volverse vector de soporte.
- Ese punto desplaza la frontera.

Ejemplo real: dataset del taller (fraude con tarjetas)

- SVC con kernel RBF: **1 074 de 1 600** transacciones de entrenamiento (67%) son vectores de soporte.
- Clases muy mezcladas → muchos puntos cerca de la frontera → muchos vectores de soporte.

Entrenamiento: un Problema de Optimización

Cómo encuentra SVM el hiperplano de margen máximo

1. Tamaño del margen

$$M = \frac{2}{\|w\|}$$

2. Objetivo: minimizar

$$\min_{w, b} \frac{1}{2} \|w\|^2$$

3. Sujeto a la restricción

$$y_i (w \cdot x_i + b) \geq 1 \quad \text{para todo } i$$

- $\|w\|$ es la longitud del vector w .
- Margen grande $\Leftrightarrow \|w\|$ pequeño: por eso se **minimiza** $\frac{1}{2}\|w\|^2$.
- y_i es la etiqueta de cada punto: +1 o -1.
- La restricción exige que todo punto quede del lado correcto y fuera del margen.

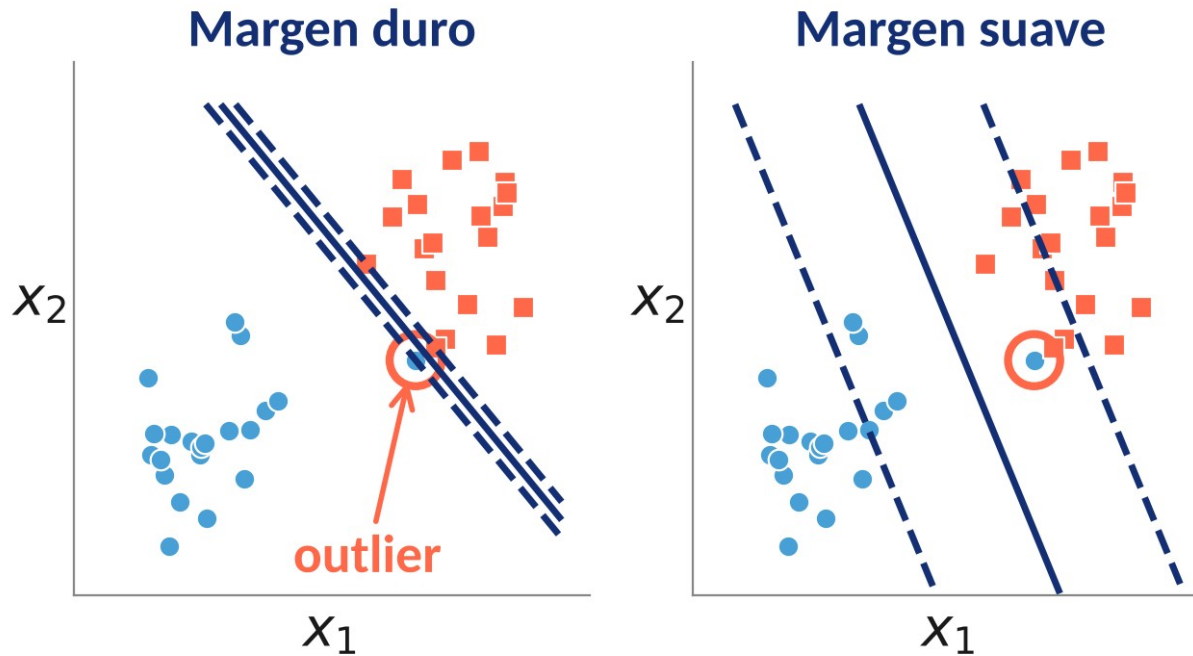
Clasificar un punto nuevo

$$\hat{y} = \text{signo}(w \cdot x + b)$$

scikit-learn resuelve este problema por nosotros al llamar fit()

Margen Duro vs. Margen Suave

Datos reales casi nunca son perfectamente separables



$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

- ξ_i (holgura) mide cuánto viola el margen cada punto; C decide cuánto se castiga esa violación.

Margen duro (hard)

- Ningún punto puede violar el margen.
- Un solo outlier lo estrecha o lo vuelve imposible.

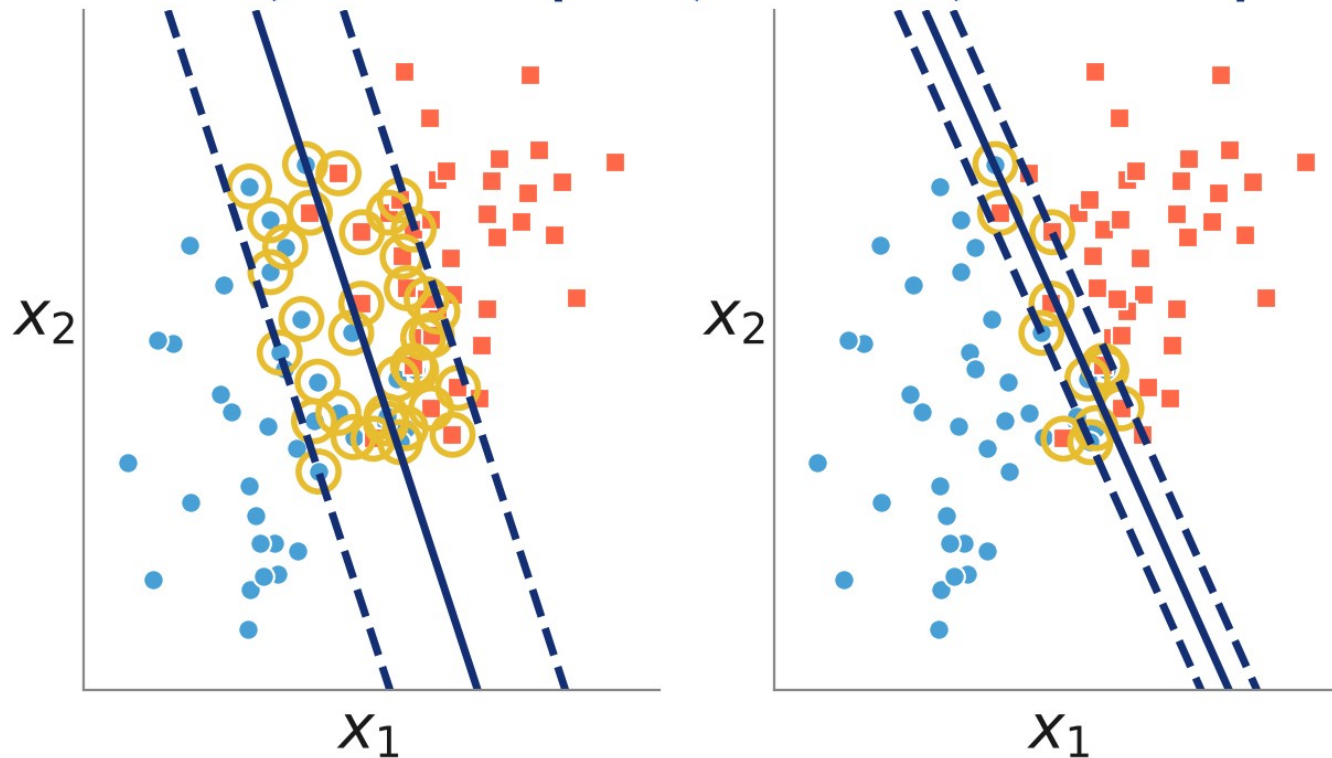
Margen suave (soft)

- Permite algunas violaciones del margen.
- Es el modo que usa scikit-learn.

El Hiperparámetro C

Controla el equilibrio entre margen amplio y pocos errores

C = 0.05 (37 vect. soporte) **C = 100 (12 vect. soporte)**



C bajo (ej. 0.05)

- Margen amplio, tolera errores.
- Muchos vectores de soporte.
- Riesgo: **underfitting** (modelo demasiado simple).

C alto (ej. 100)

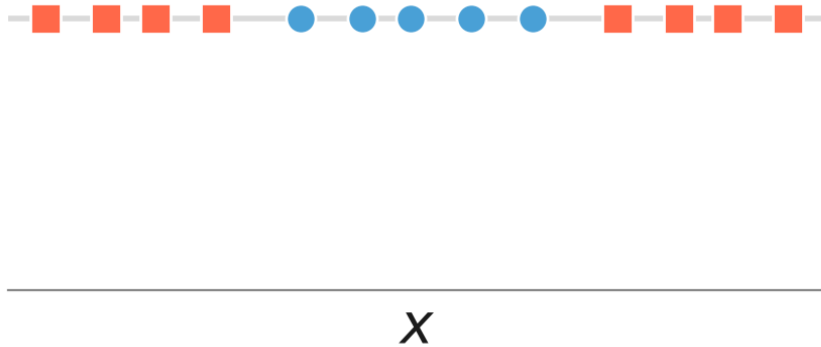
- Margen estrecho, castiga errores.
- Pocos vectores de soporte.
- Riesgo: **overfitting** (memoriza el entrenamiento).

Regla práctica: overfitting → reducir C · underfitting → aumentar C · por defecto C = 1.0

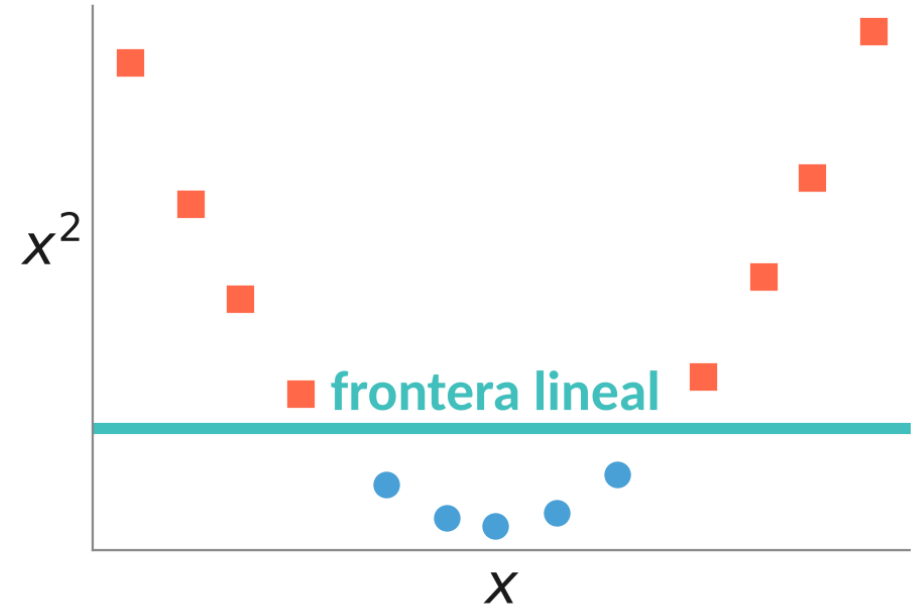
Datos No Lineales: el Truco del Kernel

Si una línea no separa las clases, se cambia el espacio donde se buscan

1. Original (1D): no separable



2. Transformado (x, x^2) : separable



Idea

- Agregar una variable nueva (aquí x^2) vuelve los datos separables por una línea.

El “truco”

- El **kernel** calcula la similitud en ese espacio nuevo sin construirlo nunca.

Kernel Lineal

El más simple: una frontera recta en el espacio original

Características

- No transforma los datos.
- Es el más rápido de entrenar.
- Se recomienda como **primer modelo**.
- Hiperparámetro principal: **C**.

Cuándo usarlo

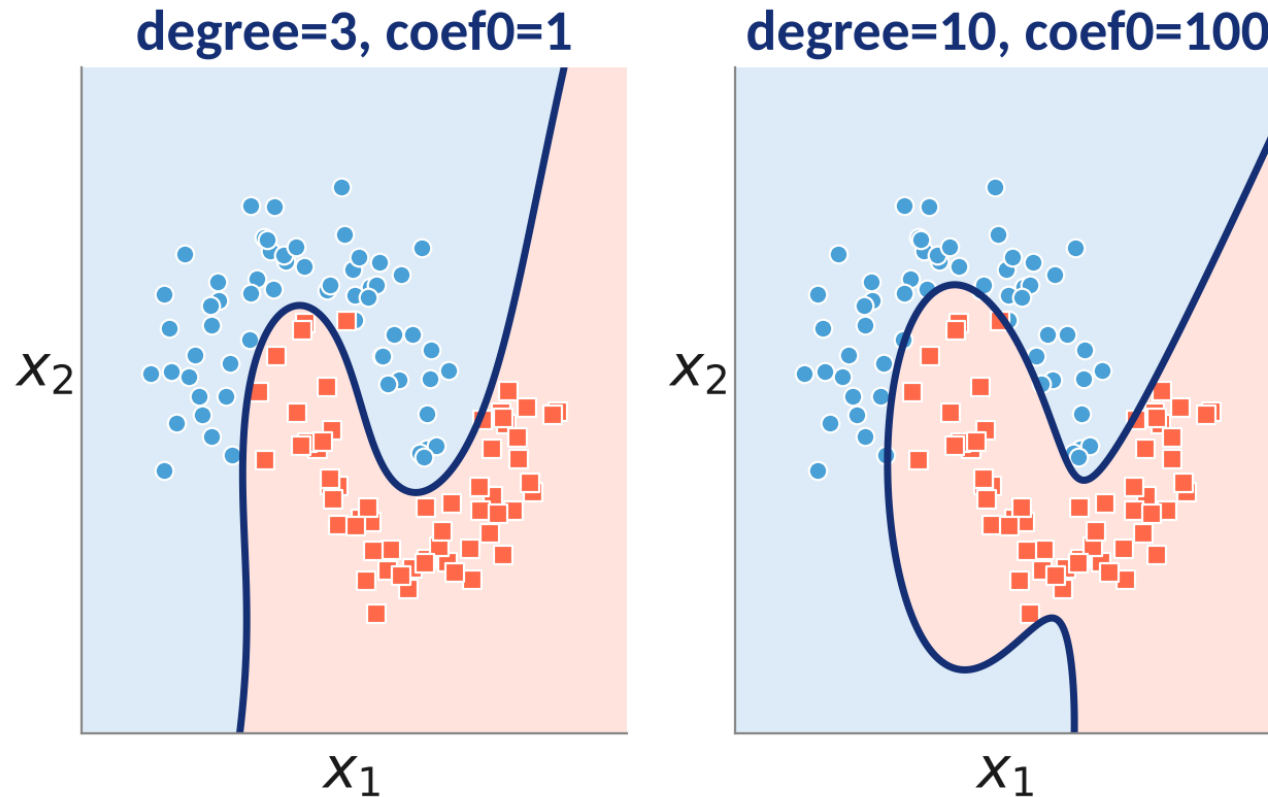
- Muchas variables (ej. texto).
- Clases que parecen separables con una recta.
- Datasets grandes.
- Como base para comparar otros kernels.

```
from sklearn.svm import SVC
modelo = SVC(kernel='linear', C=1.0)    # SVM con frontera recta
```

Nota: para datasets muy grandes (más de ~100 000 filas) existe LinearSVC, una versión más rápida del kernel lineal.

Kernel Polinómico

Fronteras curvas a partir de combinaciones polinómicas de las variables



```
SVC(kernel='poly', degree=3, coef0=1, C=5)
```

$$K(x, x') = (\gamma x \cdot x' + \text{coef0})^d$$

Parámetro	Defecto
degree (d)	3
coef0	0.0
C	1.0

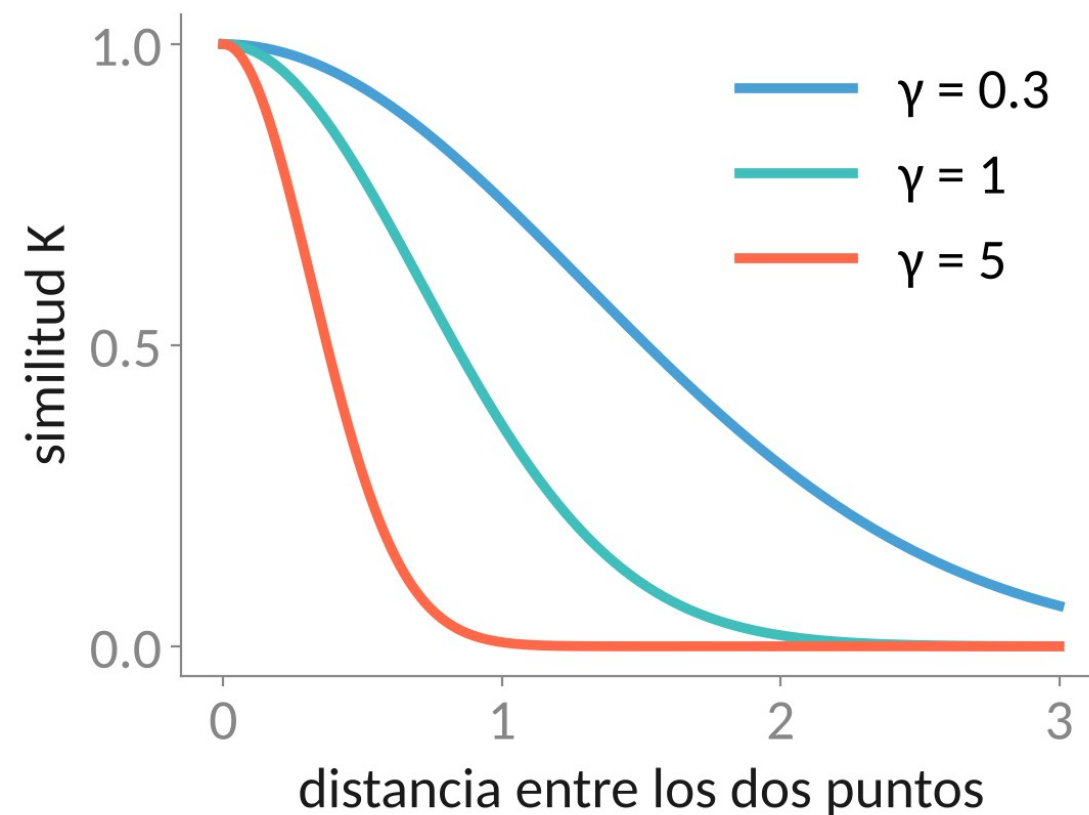
- **degree**: grado del polinomio.
- **coef0**: peso de los términos de grado bajo.
- Grado alto → frontera muy irregular (overfitting).

Kernel Gaussiano RBF (1/2)

RBF = Radial Basis Function (función de base radial): el kernel por defecto de SVC

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

- Mide la **similitud** entre dos puntos.
- Puntos cercanos $\rightarrow$ similitud ≈ 1 .
- Puntos lejanos $\rightarrow$ similitud ≈ 0 .
- **γ (gamma)** decide qué tan rápido cae la similitud con la distancia.
- Puede separar casi cualquier forma de datos.

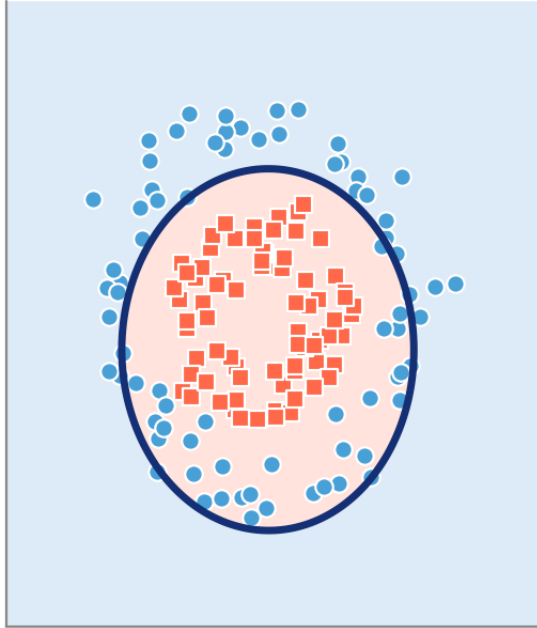


Lectura: γ grande $\rightarrow$ campana angosta $\rightarrow$ cada punto influye solo en su vecindad.

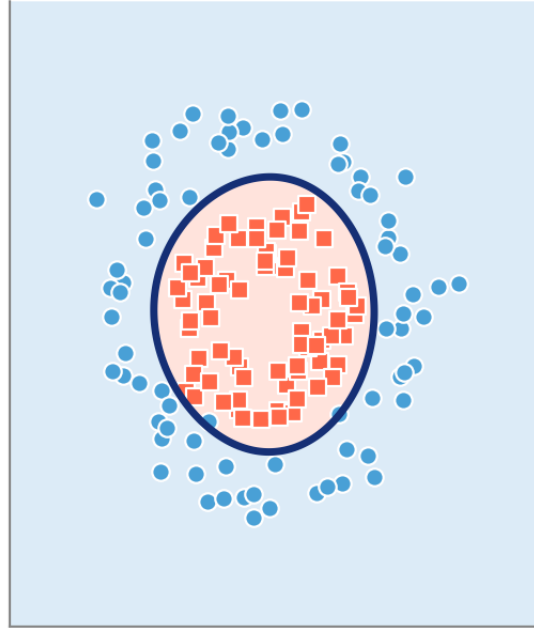
Kernel Gaussiano RBF (2/2): el Efecto de gamma

Mismo dataset, mismo $C = 1$, solo cambia gamma

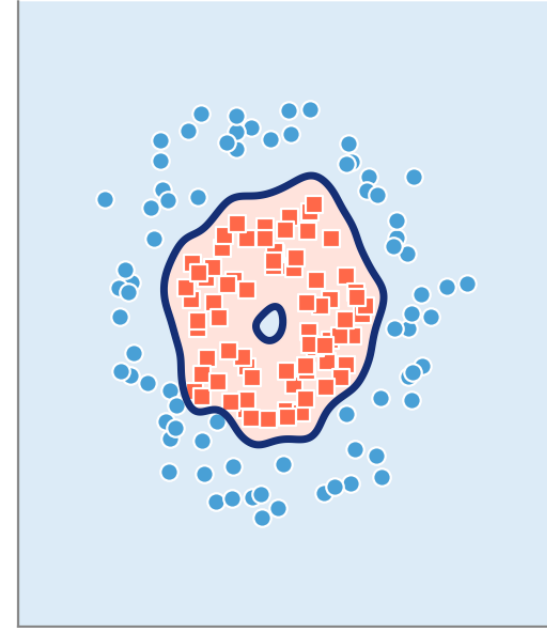
$\gamma = 0.1$: underfitting



$\gamma = 1$: equilibrio



$\gamma = 50$: overfitting



Regla práctica para gamma

- Overfitting → reducir gamma.
- Underfitting → aumentar gamma.

Por defecto: gamma='scale'

$$\gamma_{\text{scale}} = \frac{1}{n_{\text{variables}} \cdot \text{Var}(X)}$$

¿Cómo Elegir el Kernel?

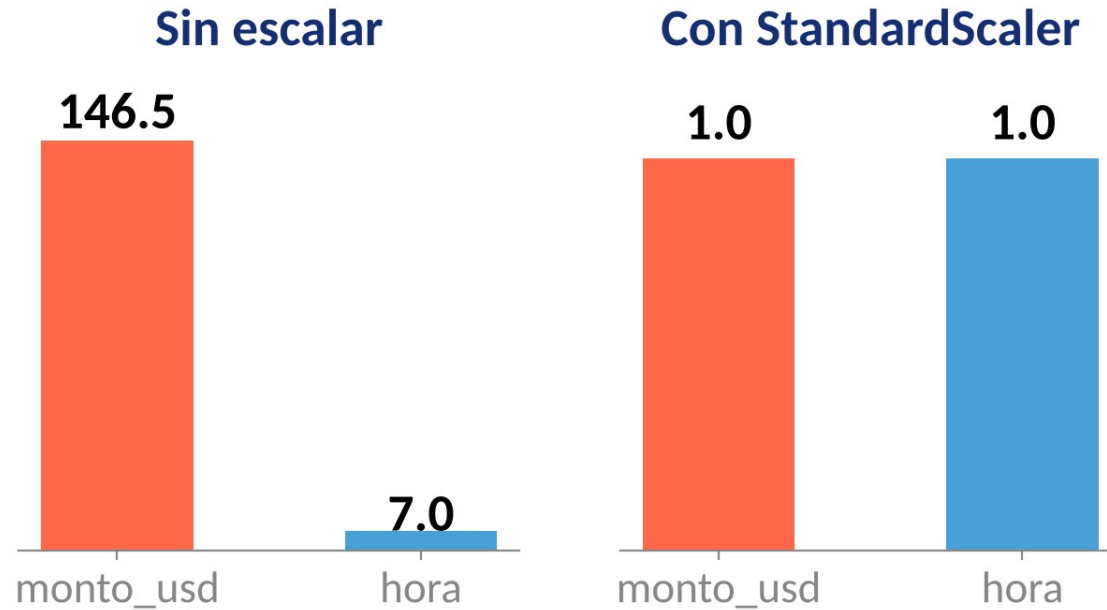
Empezar simple y aumentar la complejidad solo si las métricas lo justifican

Kernel	Código	Mejor para
Lineal	<code>SVC(kernel='linear')</code>	Muchas variables, primer modelo
RBF	<code>SVC(kernel='rbf')</code>	Fronteras no lineales; opción por defecto
Polinómico	<code>SVC(kernel='poly', degree=3)</code>	Interacciones polinómicas conocidas

- 1 Entrenar primero con kernel lineal: es la línea base.
- 2 Probar RBF con los valores por defecto ($C = 1$, $\gamma = 'scale'$).
- 3 Comparar en el conjunto de prueba con la métrica que importa al problema.
- 4 Quedarse con el más simple si la diferencia es pequeña.

Preprocesamiento Obligatorio: Escalado

SVM calcula distancias: una variable con valores grandes domina a las demás · barras = desviación estándar



- En el dataset del taller, monto_usd varía **20 veces más** que hora.
- Sin escalar, la hora casi no influye en el margen.
- **StandardScaler**: media 0 y desviación 1.

Antes de entrenar una SVM

- Resolver valores nulos y convertir categóricas a números (One-Hot Encoding).
- Ajustar el scaler **solo con X_train** y aplicarlo a X_test: así no hay fuga de datos.
- Con Pipeline esto ocurre automáticamente en `fit()` y `predict()`.

Dataset del Taller — Fraude con Tarjetas

fraude_tarjetas_ecuador.csv · 2 000 transacciones · dataset sintético · sin valores nulos

Variable	Tipo	Descripción
monto_usd	Numérica	Monto de la transacción (10 a 1 234 USD)
hora	Numérica	Hora del día (0 a 23)
tipo_comercio	Categórica	Supermercado, Restaurante, Electronica, Farmacia, Gasolinera, Viajes
dispositivo	Categórica	tarjeta_fisica, app_movil, web
historial_fraude	Binaria	1 = el titular tiene antecedentes de fraude
fraude	Objetivo (y)	1 = fraudulenta · 0 = legítima

Balance de clases

- Legítimas: **1 426** (71.3%)
- Fraudes: **574** (28.7%)

% de fraude según historial_fraude

- Sin antecedentes: **19.2%**
- Con antecedentes: **55.8%**

Clases Desbalanceadas: `class_weight='balanced'`

71% legítimas vs 29% fraudes: el modelo tiende a favorecer a la clase mayoritaria

El problema

- Cada error cuesta lo mismo para ambas clases.
- Hay 2.5 veces más legítimas: el modelo “aprende” a decir legítima.
- Resultado: se escapan muchos fraudes.

La solución

- `class_weight='balanced'` da **más peso** a los errores en la clase minoritaria.
- Equivale a usar un C mayor para la clase fraude.

$$\text{peso}_c = \frac{n_{\text{total}}}{k \cdot n_c}$$

Clase	Casos	Peso
Legítima (0)	1 426	$2000 / (2 \cdot 1426) = 0.70$
Fraude (1)	574	$2000 / (2 \cdot 574) = 1.74$

Cada error en un fraude pesa ≈ 2.5 veces más que un error en una legítima ($k = \text{número de clases} = 2$)

La Clase SVC() (1/2): Parámetros del Constructor

Se pasan al crear el modelo · SVC = Support Vector Classifier

Parámetro	Defecto	Qué controla
kernel	'rbf'	Forma de la frontera: 'linear', 'poly' o 'rbf'
C	1.0	Cuánto se castigan los errores (ancho del margen)
gamma	'scale'	Alcance de cada punto (solo kernels 'rbf' y 'poly')
degree	3	Grado del polinomio (solo kernel 'poly')
class_weight	None	'balanced' compensa clases desbalanceadas

```
from sklearn.svm import SVC
# Crear el modelo: kernel RBF + clases balanceadas
modelo = SVC(kernel='rbf', C=1.0, gamma='scale',
              class_weight='balanced')
```

La Clase SVC() (2/2): Métodos y Atributos

Lo que se puede consultar después de entrenar

Métodos (igual que en Regresión Logística)

- `fit(X, y)` entrena · `predict(X)` devuelve la clase 0 o 1.
- `decision_function(X)`: distancia con signo al hiperplano (positivo → clase 1).

`n_support_` — no es un parámetro, es un ATRIBUTO: solo existe DESPUÉS de `fit()`. Cuenta los vectores de soporte por clase.

 **SVC no tiene `predict_proba()` por defecto.** Para obtener probabilidades hay que crear `SVC(probability=True)`, que entrena más lento. En este curso usamos `predict()`.

```
modelo.fit(X_esc, y)           # X_esc: datos ya escalados
print(modelo.n_support_)       # ej.: [739 335]
```

Preparando los Datos (1/2): X, y y División

Mismo flujo que en Regresión Logística — cambia solo el modelo

```
import pandas as pd
from sklearn.model_selection import train_test_split
df = pd.read_csv('fraude_tarjetas_ecuador.csv')

# X: las 5 variables predictoras · y: la columna objetivo
X = df.drop('fraude', axis=1)
y = df['fraude']

# 80% entrenamiento / 20% prueba, misma proporción de fraudes
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

stratify=y: mantiene ≈ 28.7% de fraudes tanto en train como en test. Sin él, el azar podría dejar menos fraudes en uno de los dos conjuntos.

Preparando los Datos (2/2): ColumnTransformer

One-Hot Encoding a las categóricas + StandardScaler a las numéricas, en un solo objeto

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

cat_cols = ['tipo_comercio', 'dispositivo']
num_cols = ['monto_usd', 'hora', 'historial_fraude']

preproc = ColumnTransformer([
    ('cat', OneHotEncoder(drop='first',
                           handle_unknown='ignore'), cat_cols),
    ('num', StandardScaler(), num_cols),
])
```

- preproc todavía **no** está entrenado: se ajusta dentro del Pipeline, solo con X_train.
- handle_unknown='ignore': una categoría nunca vista no rompe la predicción.

Entrenando SVM con Pipeline

Pipeline = preproc + modelo en un solo objeto, entrenado sobre datos crudos

```
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC

pipe_rbf_bal = Pipeline([
    ('preproc', preproc),
    ('modelo', SVC(kernel='rbf', class_weight='balanced'))],
])
pipe_rbf_bal.fit(X_train, y_train)      # escala + entrena
y_pred = pipe_rbf_bal.predict(X_test)   # escala + predice
```

En el taller se entrenan 4 pipelines

pipe_lineal / pipe_rbf

pipe_lineal_bal / pipe_rbf_bal

Lo único que cambia: el paso 'modelo'

SVC(kernel='linear') / SVC(kernel='rbf')

... + class_weight='balanced'

Evaluando el Modelo: classification_report

Resultado real de pipe_rbf_bal sobre las 400 transacciones de prueba

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred,
                           target_names=['Legítima', 'Fraude']))
```

Clase	Precision	Recall	F1-score	Casos
Legítima	0.82	0.68	0.74	285
Fraude	0.44	0.62	0.51	115
Accuracy			0.66	400

Cómo leerlo (todas las métricas: rango 0 a 1 → 1 = deseado)

- **Recall de Fraude = 0.62:** de cada 100 fraudes reales, el modelo detecta 62.
- **Precision de Fraude = 0.44:** de cada 100 alertas de fraude, 44 son fraudes reales.

Resultados Reales: ¿Qué Modelo Detecta Más Fraudes?

Conjunto de prueba: 400 transacciones, 115 fraudes reales · métricas de la clase Fraude

Modelo	Accuracy	Recall	F1	Fraudes detectados	Vect. soporte
Lineal	0.72	0.51	0.51	59 de 115	859
RBF	0.74	0.37	0.44	42 de 115	872
Lineal + balanced	0.68	0.65	0.54	75 de 115	1 131
RBF + balanced	0.66	0.62	0.51	71 de 115	1 074

Conclusiones (Accuracy, Recall y F1: rango 0 a 1 → 1 = deseado)

- El modelo con **mayor Accuracy (RBF)** es el que **más fraudes deja pasar**: Accuracy no basta.
- `class_weight='balanced'` sube el Recall de fraude de forma clara en ambos kernels.
- Aquí el kernel lineal iguala o supera a RBF: **más complejo no siempre es mejor**.

SVM vs. Regresión Logística

Dos clasificadores, dos objetivos distintos

Criterio	Regresión Logística	SVM
Qué optimiza	Verosimilitud (probabilidades)	Margen entre clases
Probabilidades	Sí: predict_proba()	Solo con probability=True
Datos no lineales	Requiere crear variables	Kernels RBF o polinómico
Datasets muy grandes	Rápida	Lenta (usar LinearSVC)
Interpretación	Alta: coeficientes	Media: vectores de soporte
Hiperparámetros	Pocos (C)	Más (kernel, C, gamma, degree)
Outliers	Sensibilidad moderada	Sensibilidad alta

En la práctica: entrenar ambos y elegir con las métricas del conjunto de prueba

Ventajas y Limitaciones de SVM

Cuándo conviene usarla y cuándo no

✓ Ventajas

- Buen desempeño con pocos datos.
- Funciona con muchas variables (ej. texto).
- Kernels: fronteras flexibles.
- El margen actúa como regularización: controla el overfitting.

✗ Limitaciones

- Lenta con más de ~100 000 filas.
- Hay que ajustar kernel, C y gamma.
- Sensible a outliers y a la escala.
- Sin probabilidades por defecto.
- Difícil de interpretar con RBF.

SVM brilla con: datos pequeños o medianos · muchas variables · fronteras no lineales

Aplicaciones Reales de SVM

Casos documentados con resultados verificables

Clasificación de texto

- Joachims (1998): SVM superó a Naive Bayes, k-NN y árboles en noticias Reuters.

Diagnóstico de cáncer de mama

- 569 biopsias (UCI, Wisconsin): SVC RBF + escalado → 98.2% de accuracy en prueba (cálculo propio).

Reconocimiento de dígitos (MNIST)

- SVM con kernel gaussiano: 1.4% de error en 10 000 dígitos de prueba.

Detección de rostros

- Osuna, Freund y Girosi (1997) usaron SVM para detectar rostros en imágenes.

Fuentes: Joachims, T. (1998), ECML • LeCun, Y., MNIST database (yann.lecun.com/exdb/mnist) • UCI Breast Cancer Wisconsin (Diagnostic) • Osuna, E. et al. (1997), CVPR.

Entrenamiento Final con el 100% de los Datos

Modelo elegido: kernel lineal + class_weight='balanced' (mayor Recall y F1 de fraude)

- El split 80/20 sirvió para MEDIR el desempeño. Validado el modelo, se reentrena con todos los datos.

Modelo de evaluación (80%)

- Entrenado con X_train; solo sirvió para medir.

Pipeline final (100%)

- Entrenado con X, y completos; se exporta.

```
pipeline_final = Pipeline([
    ('preproc', preproc),
    ('modelo', SVC(kernel='linear', class_weight='balanced')),
])
pipeline_final.fit(X, y)    # X, y completos – no X_train
```

pipeline_final aprende escalado, codificación y modelo con las 2 000 transacciones

Exportación del Pipeline con joblib

joblib guarda objetos de Python ya entrenados en un archivo .pkl

```
import joblib
# Guarda preproc + modelo juntos, como un solo objeto
joblib.dump(pipeline_final, 'modelo_fraude_svm.pkl')
```

¿Por qué exportar el Pipeline completo?

- El archivo incluye las categorías aprendidas por OneHotEncoder y la media/desviación del scaler.
- Nunca se desincronizan el preprocesamiento y el modelo.
- Otro programa (ej. una API con FastAPI) solo necesita cargar este archivo.

 **No usar `pd.get_dummies` con una fila nueva.** Con una sola transacción genera columnas distintas a las del entrenamiento; el Pipeline lo evita.

Uso del Modelo con Datos Nuevos

Desde otro programa: solo se necesita el archivo .pkl, pandas, scikit-learn y joblib

```
import joblib
import pandas as pd

modelo = joblib.load('modelo_fraude_svm.pkl')

nueva = pd.DataFrame({'monto_usd': [850.0], 'hora': [2],
                      'tipo_comercio': ['Electronica'],
                      'dispositivo': ['web'],
                      'historial_fraude': [1]})

pred = modelo.predict(nueva)
print('FRAUDE' if pred[0] == 1 else 'LEGÍTIMA')    # → FRAUDE
```

predict() aplica codificación y escalado por dentro: el DataFrame va con los datos crudos